GAME LEADERBOARD & PLAYER RANKING SYSTEM

GROUP MEMBERS

*Karthik— 2510030215*Aneeq —2510080005
Abhinav —2510080002

GUIDED BY

DR.A.MaheshBabu

KL UNIVERSITY — DEPARTMENT OF CSE

PROJECT OVERVIEW

PROJECT DESCRIPTION

A **Game Leaderboard & Player Ranking System** is used to manage players, track scores, and generate rankings in online and offline games. Modern gaming platforms rely on efficient score management to provide real-time leaderboards and fair matchmaking.

This project demonstrates how **data structures and algorithms** help manage player information, rankings, and game sessions efficiently using arrays, linked lists, stacks, queues, searching, and sorting techniques.

Player Management

Store and manage player IDs, names, scores, and levels efficiently

Real-Time Rankings

Generate and update leaderboards dynamically as scores change

Fair Matchmaking

Queue-based system ensures fair and ordered game session assignments

Efficient Search

Quickly locate player records using hashing or binary search

Made with GAMMA

DATA STRUCTURES — PART 1

ARRAYS & LINKED LISTS

1. ARRAYS

Arrays are used to store **fixed player-related data** such as:

- Player IDs
- Player names
- Current scores
- Player levels

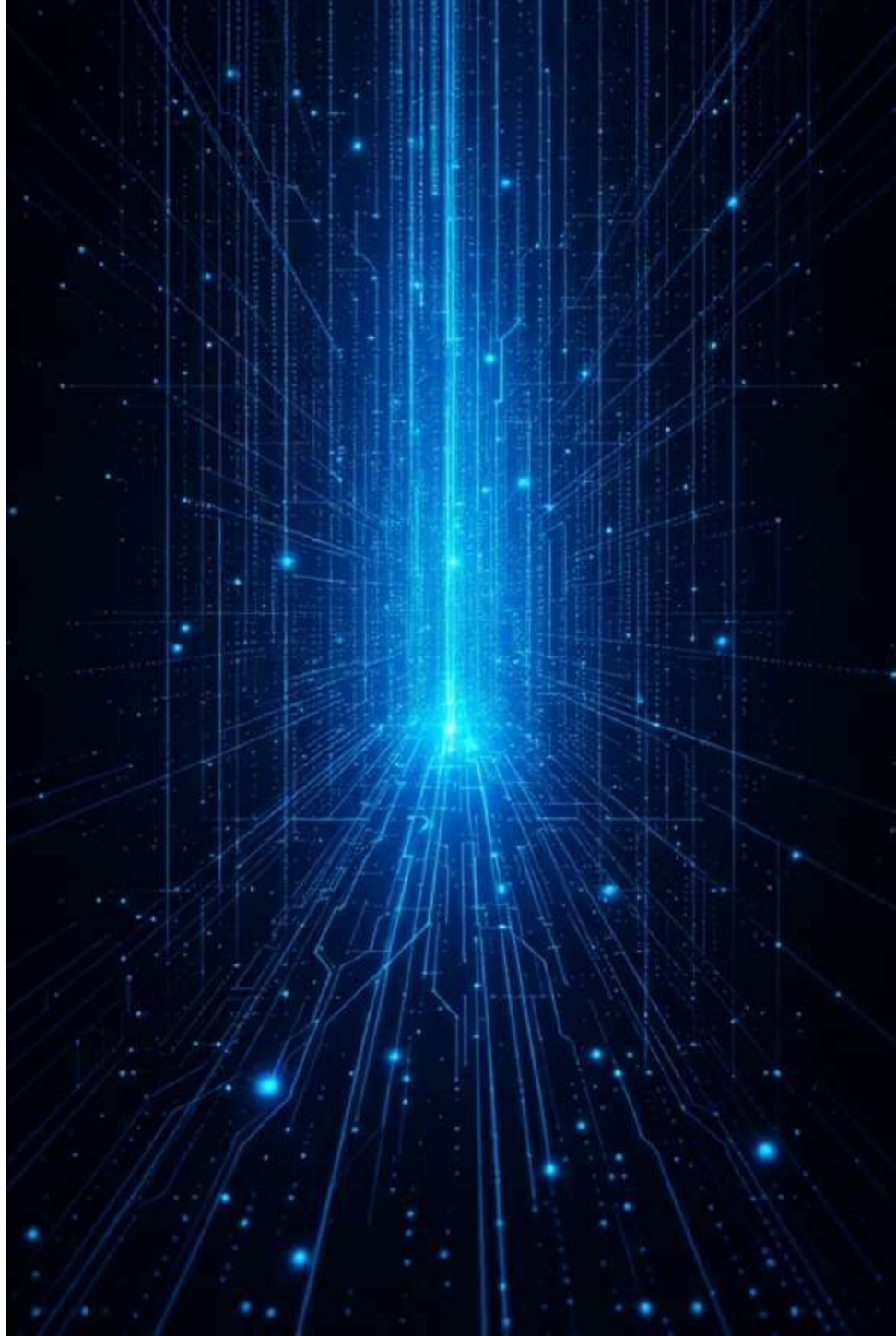
Arrays provide **fast access** and efficient indexing.

2. LINKED LISTS

Linked lists are used for **dynamic data** such as:

- Top-ranked players list
- Recently active players
- Tournament participant lists

They allow **flexible updates** as rankings change frequently.



DATA STRUCTURES — PART 2

STACKS & QUEUES

3. STACK

Stacks are used for:

- Undoing the last score update
- Reverting player rank changes

Stacks help **recover from accidental updates** using the LIFO principle.

4. QUEUE

Queues are used for:

- Matchmaking requests
- Players waiting to join a game

Queues ensure **fair FIFO-based matchmaking** for all players.

Made with GAMMA

ALGORITHMS — PART 3

SEARCHING & SORTING ALGORITHMS

5. SEARCHING ALGORITHMS

Searching algorithms help locate:

- Player details using Player ID
- Score records across the system

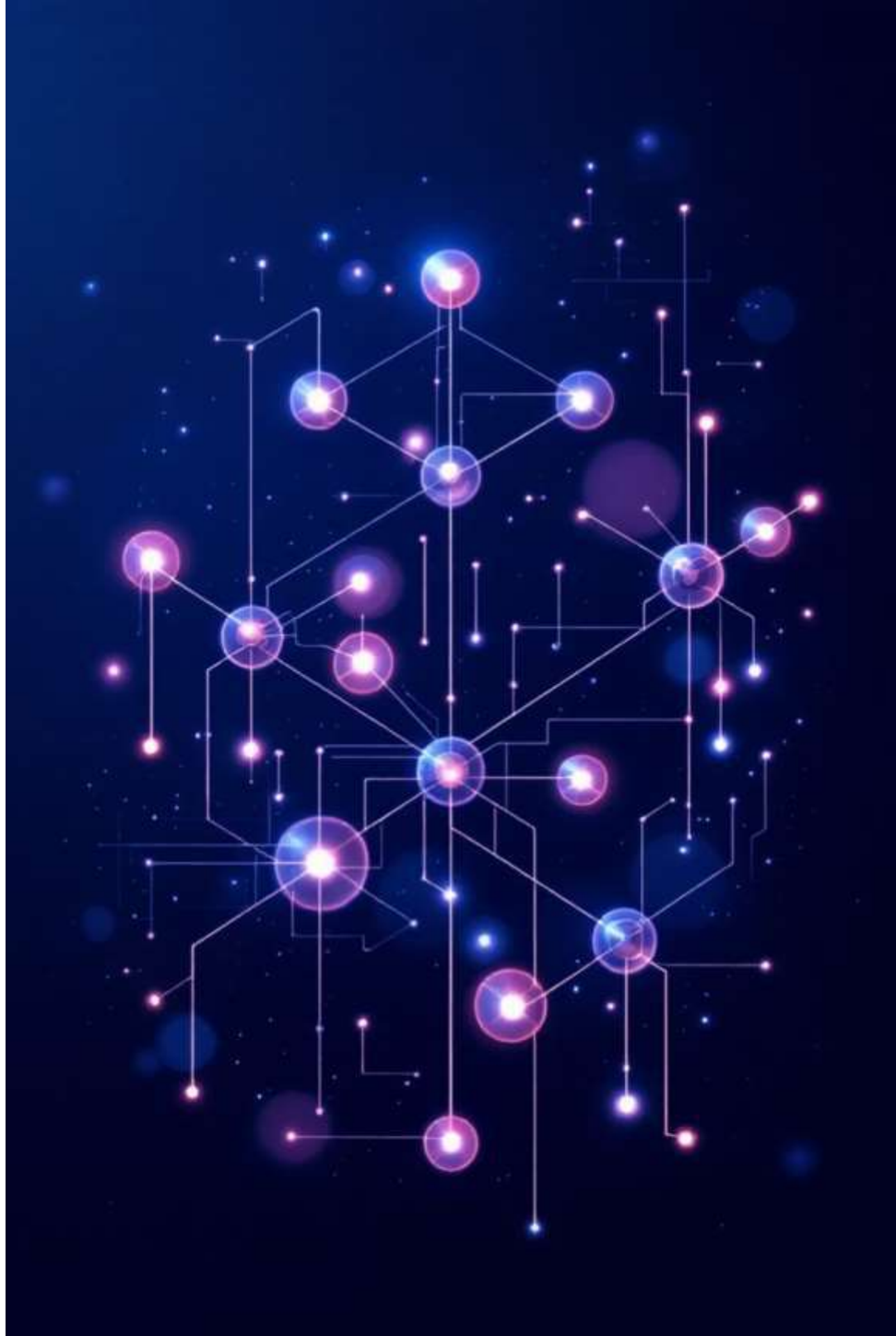
Hashing or Binary Search improves search efficiency significantly.

6. SORTING ALGORITHMS

Sorting is used for:

- Ranking players by score
- Displaying leaderboards in order

Efficient sorting improves competitive gameplay and fairness.



IMPLEMENTATION

OPERATIONS — CODE & OUTPUT

The following section demonstrates the core operations of the Game Leaderboard & Player Ranking System with code screenshots and corresponding output.

Codes

```java
package main;


public class Leaderboard {
    private static class Node {
        String  name;
        int     score;
        boolean isActive;
        boolean isTournament;
        Node    next;

        Node(String name, int score, boolean isActive, boolean isTournament) {
            this.name         = name;
            this.score        = score;
            this.isActive     = isActive;
            this.isTournament = isTournament;
            this.next         = null;
        }
    }

    PlayerMan[] topPlayersList = new PlayerMan[5];     //calling array of objects which is the data
    private Node head;  // start of the linked list


    public void generateLeaderboard(PlayerMan[] allPlayers) {
    }

    private void buildLinkedList() {
    }

    public void displayTop5() {
    }

    public void displayActive() {
    }

    public void displayTournament() {
    }
}
```

```java
package main;


public class ActionStack {

    // ==================================================================
    // PART 1 — Inner Node class
    // ==================================================================

    private static class Node {
    }

    // ==================================================================
    // PART 2 — Fields
    // ==================================================================

    private Node top;  // the most recently pushed node (top of the stack)
    private int  size; // how many snapshots are stored

    private static final int MAX_HISTORY = 20; // cap to avoid infinite memory use

    // ==================================================================
    // PART 3 — Constructor
    // ==================================================================

    public ActionStack() {
    }

    // ==================================================================
    // PART 4 — Core Stack Operations
    // ==================================================================


    public void push(PlayerMan[] players) {
    }

    public PlayerMan[] pop() {
    }

    public PlayerMan[] peek() {
    }

    public boolean isEmpty() { return top == null; }
    public int     getSize() { return size; }

    public void clear() {
    }

    // ==================================================================
    // PART 5 — Helpers
    // ==================================================================
```

Made with GAMMA

```java
package main;
import java.util.Scanner;


public class Matchmakingqueue {

    // ==================================================================
    // PART 1 — Inner Node class
    // ==================================================================
    private static class Node {
    }

    // ==================================================================
    // PART 2 — Fields
    // ==================================================================

    private Node head;  // FRONT of the queue — dequeue happens here
    private Node tail;  // BACK  of the queue — enqueue happens here
    private int  size;  // how many players are currently waiting

    // ==================================================================
    // PART 3 — Constructor
    // ==================================================================

    public Matchmakingqueue() {
    }

    // ==================================================================
    // PART 4 — Core Queue Operations
    // ==================================================================

    //ADDING PLAYERS TO THE QUEE
    public void enqueue(PlayerMan player) {
    }

    //AFTER MATCHMAKING REMOVEING THEM FROM QUEUE+
    private PlayerMan dequeue() {
    }

    public PlayerMan peek() {
    }

    public boolean isEmpty() { return size == 0; }
    public int     getSize() { return size; }

    // ==================================================================
    // PART 5 — Remove a specific player from the queue (by ID)
    // ==================================================================

    public void removeFromQueue(int id) {
    }
```

```java
package main;

import java.util.Scanner;

public class PlayerMan {
    int id;
    String name;
    int score;
    boolean isActive;
    boolean isTournament;

    // Constructor
    public PlayerMan(int id, String name, int score, boolean isActive, boolean isTournament) {
    }

    //Add Player
    public static PlayerMan[] addPlayer(PlayerMan[] oldArray, Scanner sc) {
    }

    //Update data
    public static void updatePlayer(PlayerMan[] players, Scanner sc) {
    }

    //Delete Player
    public static PlayerMan[] deletePlayer(PlayerMan[] oldArray, Scanner sc) {
    }

    //Display All
    public static void display(PlayerMan[] players) {
    }

    //Display Active Players
    public static void displayActive(PlayerMan[] players) {
    }

    //Display Tournament Players
    public static void displayTournament(PlayerMan[] players) {
    }


    //******************
    //ERROR HANDLING
    //******************

    //(Integer Values)
    public static int getValidInt(Scanner sc, String prompt) {
    }

    //(for boolean values)
    public static boolean getBooleanInput(Scanner sc, String prompt) {
    }
```

```java
package main;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        PlayerMan[]      players     = new PlayerMan[0];
        Leaderboard      leaderboard = new Leaderboard();
        Matchmakingqueue queue       = new Matchmakingqueue();
        ActionStack      undoStack   = new ActionStack(); // past states
        ActionStack      redoStack   = new ActionStack(); // future states (after undo)

        int mainChoice;

        do {
        } while (mainChoice != 0);

        sc.close();
    }

    // ------------------------------------------------------------------
    // Menu printers
    // ------------------------------------------------------------------

    private static void printMainMenu() {
    }

    private static void printPlayersMenu(ActionStack undoStack, ActionStack redoStack) {
    }

    private static void printLeaderboardMenu() {
    }

    private static void printMatchmakingMenu() {
    }
}
```

Made with GAMMA

Output

```
PLAYER MANAGEMENT SYSTEM

1. Players Menu
2. Leaderboard Menu
3. Matchmaking Menu
0. Exit

Choose an option: 1

PLAYERS MENU

1. Add Player
2. Update Player
3. Delete Player
4. Display All Players
5. Undo ( 0 state(s) available)
6. Redo ( 0 state(s) available)
0. Back to Main Menu

Choose an option: 1

Enter ID: 123
Enter Name: dfvg
Enter Score: 1234
Is Active? (y/n): y
Is Tournament Player? (y/n): y
Player added.
```

Made with GAMMA